CHAPTER 13

Design A Chat System

1-on-1 and group messaging with presence, multi-device sync and real-time delivery (WhatsApp / Messenger).

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a chat application supporting **1-on-1 chat, small group chat (up to 100 people), online presence, multiple device support, and push notifications**. Messages must arrive in near real time and be ordered, and history must sync across devices.

Why interviewers ask it: Tests real-time protocols (WebSocket), stateful servers, message ordering/IDs, storage choice and fan-out.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
1-on-1 or group?	Both; group max 100 members.
Mobile / web?	Both.
Scale?	50 M DAU.
Message size limit?	Text under 100,000 characters.
Extras?	Online indicator, multi-device, push notifications; no end-to-end encryption in scope.
History retention?	Forever.

3. Requirements

Functional

- Send/receive 1-on-1 and group messages in real time.
- Online/offline presence.
- Multi-device sync.
- Push notifications when offline.

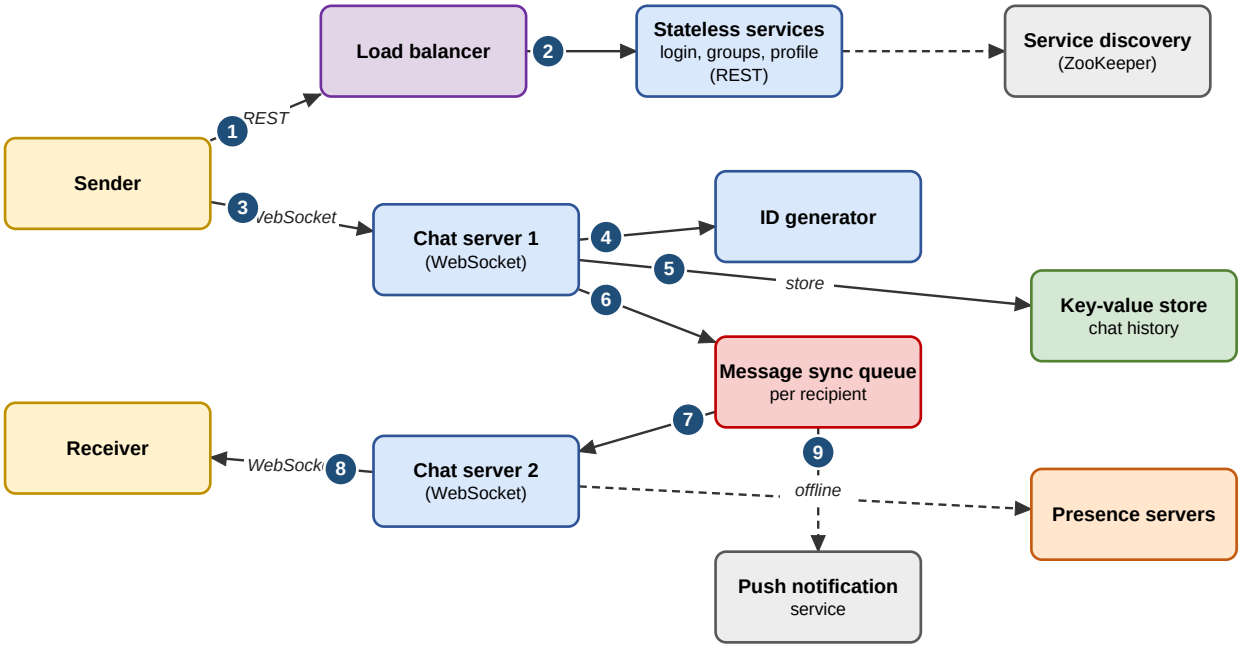
Non-functional

- Low-latency delivery (< 500 ms).
- Highly available; no message loss.
- Per-chat ordering of messages.
- Scales to 50 M DAU.

4. Back-of-the-envelope estimation

- 50 M DAU x 40 msgs/day = 2 B msgs/day = about **23 K msgs/s** (peak 2-3x).
- 100 bytes/msg -> 200 GB/day -> about 73 TB/year -> sharded KV store, hot/cold tiering.
- Concurrent connections: assume 10 M simultaneous WebSockets; each server holds about 100 K -> **100+ chat servers**.

5. High-level design



Stateful chat servers hold persistent connections; everything else is stateless behind the load balancer.

Step	What happens
1	On login the client calls a **REST** endpoint through the load balancer (authentication, profile, group management are stateless services).
2	The API tier authenticates the user and asks **service discovery** which chat server is best (by location and load).
3	The client opens a persistent **WebSocket** to chat server 1 and keeps it alive with heartbeats.
4	The sender sends a message. Chat server 1 asks the **ID generator** for a time-sortable message ID (per-chat ordering).
5	The message is persisted to the **key-value store** (chat history).
6	It is also put on the recipient's **message sync queue** (one inbox per recipient; for a group of up to 100, a copy per member).
7	If the receiver is online, chat server 2 (which holds the receiver's WebSocket) takes the message from the queue.
8	Chat server 2 pushes the message down the receiver's WebSocket.
9	If the receiver is **offline**, the message stays in the sync queue and a push notification is sent through APNs/FCM.

Request flow

- **1-on-1:** User A sends via WebSocket to chat server 1 -> server 1 asks the ID generator for a message ID -> puts the message on the recipient's sync queue and stores it in the KV store -> if B is online, server 2 (B's connection) pushes it; if offline, push notification service fires.
- **Group:** copy the message into each member's message-sync queue (inbox). Works well for small groups (<= 100): each client reads only its own queue.
- **Presence:** on login/logout and via **heartbeat** (client pings every ~5 s; if no heartbeat for ~30 s mark offline). Fan-out presence changes only to friends/group members (publish-subscribe channels).

6. Deep dive

Choosing the transport

- HTTP (keep-alive) works for senders but the **receiver** needs server push: **polling** (wasteful), **long polling** (connection churn, sticky), or **WebSocket** (bidirectional, persistent) - choose WebSocket.
- Sender can also use WebSocket so one connection serves both directions; REST handles non-real-time features (sign-up, profile).

Stateless vs stateful services

- Stateless: auth, group management, profile - behind a LB; discoverable through **service discovery** (recommend the best chat server for a client by location/capacity).
- Stateful: chat servers (connection affinity). Do not over-scale a single server; plan for graceful draining.

Storage choice

- Two data types: generic (profiles, settings, friends) -> relational DB with replication/sharding; **chat history** -> huge, mostly recent reads, random access for search/jump -> **key-value store** (HBase at Facebook, Cassandra at Discord) for horizontal scale, low latency, long-tail access.
- 1-on-1 table: `message_id` (PK), `message_from`, `message_to`, `content`, `created_at`. Group: composite key `(channel_id, message_id)` partitioned by channel.

Message ID (ordering)

- IDs must be unique and **sortable by time** so newer = larger. `auto_increment` does not exist in NoSQL -> use a **Snowflake-like** generator (chapter 08) or local sequence numbers per chat (sufficient because ordering matters only within a chat).

Service discovery and connection routing

- Apache ZooKeeper registers chat servers; on login the API returns the best server's address. Clients then open a WebSocket to it.
- A user->server mapping in a session store lets other chat servers route messages to the right connection (or use pub/sub).

Multi-device support

- Each device has its own WebSocket and `cur_max_message_id`. A device syncs messages with ID > its pointer. All devices of a user share the same sync queue/stream.

Online presence

- Presence servers keep status in Redis; heartbeats extend a TTL. Status changes published to a per-friend-pair channel (fine for small groups, throttle or lazy-fetch for large ones).

Scaling extras

- Media: upload to object storage + CDN; send only a URL in the chat message. Search: index messages (Elasticsearch). Compression, batching.
- End-to-end encryption, message edit/delete, read receipts are common follow-ups.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Transport	Polling / Long polling / WebSocket	WebSocket for real-time bi-directional; fallback to long polling.
Group fan-out	Write to each member inbox / read from shared group log	Per-member inbox for small groups; shared log for huge channels.
History store	SQL / Key-value store	KV store for massive, append-heavy chat data; SQL for relational metadata.
Presence updates	Push to all / on-demand pull	Push to small groups; pull for large groups.

8. Scaling, reliability, failure modes

- Shard KV store by (user_id or channel_id); replicate across zones.
- Chat servers can crash: clients reconnect with exponential backoff and re-sync from last message ID (no data loss since messages are in the queue/DB).
- Backpressure and connection limits; heartbeats detect dead connections.

9. Likely follow-up questions

- How do you guarantee message ordering and exactly-once display?
- How would you support 100,000-member channels?
- How do you add end-to-end encryption?

- How do you detect that a user is truly offline vs temporarily disconnected (tunnel)?

10. Common mistakes

- Using only HTTP polling at this scale.
- Global ordering requirement (unnecessary and expensive) instead of per-chat ordering.
- Not planning for stateful server failure and reconnect storms.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"I'll support 1-on-1 and group chat up to 100 people, web and mobile, 50 M DAU, text messages under 100,000 characters, online presence, multi-device sync, and push notifications. That is around 23,000 messages per second, so the main requirements are low-latency delivery, no message loss and ordering within a chat."
2. High-level design	10 min	"Clients keep a WebSocket to a chat server for real-time send and receive, while sign-up, profile and group APIs are stateless REST services behind a load balancer. Service discovery assigns a chat server. A message goes through an ID generator, is stored in a key-value store and placed on the recipient's sync queue; offline users get a push notification."
3. Deep dive	25 min	"I choose WebSocket over polling or long polling because it is bidirectional and persistent. For storage I use a key-value store for chat history because the data is huge and access is mostly recent with random lookups. Message IDs must be sortable by time, so I use a Snowflake-style generator or per-chat sequence numbers. Multi-device sync uses a per-device last-seen message ID. Presence uses heartbeats with a timeout and publishes only to friends."
4. Wrap up	4 min	"Failures: if a chat server dies, clients reconnect with backoff and resync from their last message ID. I'd add media via object storage and CDN, search with Elasticsearch, and discuss end-to-end encryption and large channels as extensions."

12. 60-second recap / practice checklist

- WebSocket chat servers + stateless REST services + service discovery.
- KV store for history; Snowflake / per-chat sequence IDs; per-recipient sync queue; heartbeat presence.
- Offline -> push notification; media via object storage/CDN.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Discord: How Discord stores billions of messages
• RFC 6455: The WebSocket protocol
• GeeksforGeeks: Design a chat application (WhatsApp)